Міністерство освіти і наук України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій
Кафедра оптоелектроніки та інформаційних технологій

Звіт
До лабораторної роботи №1
Варіант 11
На тему
"Основи функціонального програмування у Python"

Виконав:
Студентка групи ФЕП-21
Муковоз М. Т.
Перевірив:
Ас. Остріков Д. А.

Львів 2026

Мета роботи:

- зрозуміти, що таке чиста функція і референтна прозорість;
- навчитися виявляти та ізолювати побічні ефекти;
- переписати імперативний фрагмент у функціональному стилі;
- застосувати typing.Callable для параметризації обчислень

Теоретичні відомості:

• Чиста функція: детермінована (однаковий вхід → однаковий вихід), без змін глобального стану/вхідних даних, без I/O усередині.
• Референтна прозорість: вираз можна замінити його значенням без зміни поведінки програми.
• Побічні ефекти: I/O (print, файл, мережа), мутація вхідних структур, використання/зміна глобалів, залежність від time, random, os.environ тощо.
• Підхід: відокремити чисте ядро (перетворення даних) від оболонки ефектів (читання/запис).

Хід роботи

Під час виконання лабораторної роботи було створено програмне рішення для розрахунку місячної вартості послуг мобільного зв'язку. Архітектура проєкту побудована на засадах функціонального програмування з чітким відокремленням обчислювальної логіки від побічних ефектів. Основні математичні операції інкапсульовано у модулі core.py у вигляді чистих функцій. Вони генерують нові записи з підсумковою вартістю (monthly_cost), повністю уникаючи мутації оригінальних даних про використані абонентом хвилини, SMS та інтернет-трафік.

Щоб зробити систему максимально гнучкою, процеси було параметризовано за допомогою typing.Callable. Це дало змогу динамічно передавати правила тарифікації та алгоритми нарахування бонусів як окремі функції-аргументи, не "зашиваючи" їх у ядро. Усі процеси вводу-виводу інформації (зокрема, відображення фінального звіту на екрані) строго локалізовані в оболонці app.py.

Надійність реалізованого підходу доведена за допомогою автоматизованого тестування у середовищі pytest (файл test_core.py). Написані тести підтверджують референтну прозорість функцій, відсутність прихованої зміни початкового стану колекцій та точність розрахунків за різних умов. Підсумковий аудит коду показав успішне проходження всіх тестів, стабільну роботу програми та відсутність помилок статичної типізації при перевірці аналізатором mypy.

Файл core.py

core.py > with_cost

```python
from typing import Callable, TypedDict, List, Dict

class Usage(TypedDict):
    id: int
    minutes: int
    sms: int
    traffic_mb: float
    tariff: str

class BilledUsage(TypedDict):
    id: int
    minutes: int
    sms: int
    traffic_mb: float
    tariff: str
    monthly_cost: float

TariffFn = Callable[[int, int, float], float]
BonusFn = Callable[[float], float]

def with_cost(usage: Usage, cost: float) -> BilledUsage:
    """Створює новий запис з доданою ціною (без мутації вхідного словника)"""
    new_usage = dict(usage)
    new_usage["monthly_cost"] = cost
    return new_usage

def make_processor(
    calculate_base_cost: TariffFn,
    apply_bonus: BonusFn
) -> Callable[[List[Usage]], Dict[str, object]]:
    """Фабрика, яка створює готову функцію для обробки даних"""

    def process(usages: List[Usage]) -> Dict[str, object]:
        processed_records = []
        total_revenue = 0.0

        for u in usages:
            base_cost = calculate_base_cost(u["minutes"], u["sms"], u["traffic_mb"])

            final_cost = apply_bonus(base_cost)

            new_u = with_cost(u, final_cost)
            processed_records.append(new_u)

            total_revenue += final_cost

        return {
            "count": len(processed_records),
            "revenue": total_revenue,
            "records": processed_records
        }

    return process
```

Файл app.py

app.py > main

```python
from core import make_processor, Usage

def main():
    usages: list[Usage] = [
        {"id": 1, "minutes": 100, "sms": 20, "traffic_mb": 1024, "tariff": "Basic"},
        {"id": 2, "minutes": 500, "sms": 100, "traffic_mb": 5000, "tariff": "Pro"},
        {"id": 3, "minutes": 10, "sms": 0, "traffic_mb": 100, "tariff": "Basic"}
    ]

    def my_tariff(minutes: int, sms: int, traffic_mb: float) -> float:
        return (minutes * 1.0) + (sms * 0.5) + (traffic_mb * 0.1)

    def my_bonus(cost: float) -> float:
        if cost > 100:
            return cost * 0.9
        return cost

    processor = make_processor(
        calculate_base_cost=my_tariff,
        apply_bonus=my_bonus
    )

    result = processor(usages)

    print("--- Розшифровка мобільного тарифу за місяць ---")
    for record in result["records"]:
        print(f"Абонент #{record['id']} | Тариф: {record['tariff']} | До оплати: {record['monthly_cost']:.2f} грн")

    print("-" * 45)
    print(f"Оброблено абонентів: {result['count']}")
    print(f"Загальний дохід компанії: {result['revenue']:.2f} грн")

if __name__ == "__main__":
    main()
```

Файл test_core.py

```python
from core import make_processor
import copy

sample_usages = [
    {"id": 1, "minutes": 10, "sms": 5, "traffic_mb": 100, "tariff": "Test"}
]

dummy_tariff = lambda m, s, mb: m + s + (mb * 0.1)
dummy_bonus = lambda cost: cost

def test_referential_transparency():
    """Тест: однаковий вхід дає однаковий вихід"""
    processor = make_processor(dummy_tariff, dummy_bonus)

    result1 = processor(sample_usages)
    result2 = processor(sample_usages)

    assert result1 == result2

def test_no_mutation():
    """Тест: вхідні дані не змінюються"""
    processor = make_processor(dummy_tariff, dummy_bonus)
    original_data = copy.deepcopy(sample_usages)

    processor(sample_usages)

    assert sample_usages == original_data

def test_callable_policies():
    """Тест: перевірка правильності обчислень"""
    processor = make_processor(dummy_tariff, dummy_bonus)
    result = processor(sample_usages)

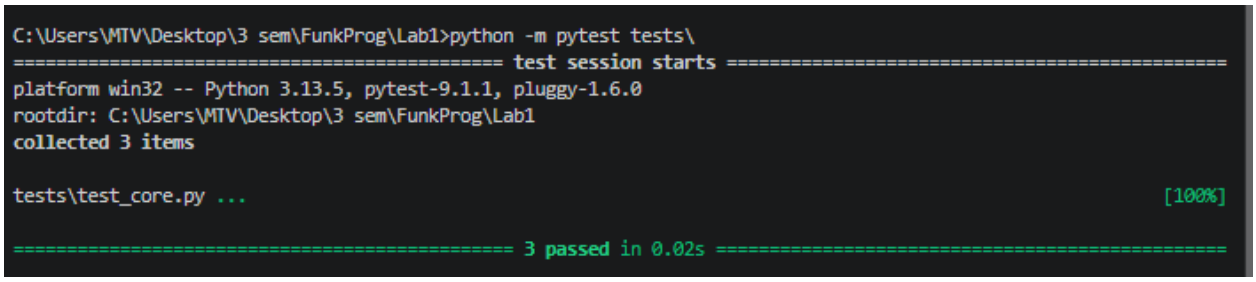
    records = result["records"]
    assert len(records) == 1
    assert records[0]["monthly_cost"] == 25.0
    assert result["revenue"] == 25.0
```

Запуск програми

```
MTV/Desktop/3 sem/FunkProg/Lab1/app.py"
--- Розшифровка мобільного тарифу за місяць ---
Абонент #1 | Тариф: Basic | До оплати: 191.16 грн
Абонент #2 | Тариф: Pro | До оплати: 945.00 грн
Абонент #3 | Тариф: Basic | До оплати: 20.00 грн
-----------------------------------------
Оброблено абонентів: 3
Загальний дохід компанії: 1156.16 грн

C:\Users\MTV\Desktop\3 sem\FunkProg\Lab1>
```

Виконання тестів



Висновок: Під час виконання лабораторної роботи початковий код на Python було успішно переписано з використанням функціонального стилю. На практиці було опрацьовано створення чистих функцій та відокремлення математичних розрахунків від виведення інформації на екран (I/O). Такий підхід дозволив повністю прибрати побічні ефекти з головної логіки. Завдяки інструменту typing.Callable програма стала гнучкішою: правила розрахунків тепер передаються як звичайні аргументи. У результаті розроблений код не використовує глобальних змінних і не змінює початкових даних, що робить його надійним та референтно прозорим.